МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА — Российский технологический университет»
Институт технологий искусственного интеллекта
Кафедра _____

КУРСОВАЯ РАБОТА

по дисциплине «_____»

на тему:

«Реализация алгоритма вычисления расстояния редактирования (Левенштейна) с оптимизацией памяти и параллельным вычислением»

Выполнил:

студент группы _____

_____

Руководитель:

_____

Москва, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 3
1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ 5
1.1 Понятие редакционного расстояния 5
1.2 Классический алгоритм Вагнера — Фишера 6
1.3 Оптимизация памяти 7
1.4 Способы распараллеливания вычислений 8
1.5 Постановка задачи на разработку 10
1.6 Выводы по разделу 1 11
2 РАЗРАБОТКА АЛГОРИТМА И ЕГО ПРОГРАММНАЯ РЕАЛИЗАЦИЯ 12
2.1 Выбор средств разработки 12
2.2 Структура программы 12
2.3 Разработка алгоритма 13
2.4 Реализация классического алгоритма 14
2.5 Реализация с оптимизацией памяти 15
2.6 Реализация параллельных вычислений 16
2.7 Тестирование программы 17
2.8 Результаты экспериментальных замеров 17
2.9 Выводы по разделу 2 21
ЗАКЛЮЧЕНИЕ 22
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 24
ПРИЛОЖЕНИЕ А 26
ПРИЛОЖЕНИЕ Б 42
ПРИЛОЖЕНИЕ В 44

ВВЕДЕНИЕ

Современные информационные системы работают с большими объёмами текстовых данных: поисковыми запросами, словарями, результатами распознавания речи и изображений, научными публикациями, медицинскими записями. Во многих задачах требуется не точное, а приближённое сравнение строк, когда нужно определить, насколько одна строка похожа на другую и сколько ошибок их различает [3].

Одной из базовых мер сходства строк является расстояние Левенштейна — минимальное число операций вставки, удаления и замены одного символа, необходимых для превращения одной строки в другую [3]. Оно применяется в системах проверки орфографии, при поиске по словарю, в антиплагиате, при сравнении биологических последовательностей и при оценке качества распознавания текста [3, 8].

Классический алгоритм вычисления расстояния Левенштейна основан на методе динамического программирования и был систематизирован Р. Вагнером и М. Фишером в 1974 году [2]. Алгоритм заполняет матрицу размером $(m + 1) \times (n + 1)$, поэтому требует $O(n \cdot m)$ времени и $O(n \cdot m)$ памяти [3]. Для строк длиной 10 000 символов такая матрица содержит 100 млн чисел и занимает около 400 МБ, что делает классическую реализацию неприменимой для длинных строк. Отсюда вытекает первое направление совершенствования алгоритма — сокращение используемой памяти.

Второе направление связано с развитием вычислительной техники: современные процессоры содержат несколько ядер, и последовательная программа не использует их возможности. Однако клетки матрицы расстояний зависят друг от друга, поэтому распараллеливание алгоритма требует специальных приёмов и анализа накладных расходов на синхронизацию [14, 15].

Таким образом, тема работы актуальна: она объединяет две практические потребности — экономию памяти при обработке длинных

строк и использование многоядерности процессора для ускорения вычислений.

Объект исследования — процесс вычисления редакционного расстояния между строковыми последовательностями.

Предмет исследования — алгоритм Левенштейна и его программная реализация с оптимизацией памяти и параллельными вычислениями на языке Java.

Цель работы — разработать и программно реализовать алгоритм вычисления расстояния Левенштейна с оптимизацией памяти и параллельным вычислением.

Для достижения цели необходимо решить следующие задачи:

– проанализировать литературу по теме и существующие подходы к вычислению редакционного расстояния;

– выбрать и обосновать метод оптимизации памяти и способ распараллеливания;

– разработать структуру программы и реализовать алгоритм на языке Java;

– провести тестирование корректности всех реализаций;

– выполнить экспериментальные замеры времени и памяти и проанализировать результаты.

Методы исследования: анализ научной и технической литературы, метод динамического программирования, объектно-ориентированное проектирование, тестирование программного обеспечения, экспериментальные измерения времени и объёма памяти.

Работа состоит из введения, двух разделов, заключения, списка использованных источников и трёх приложений. В первом разделе рассматриваются теоретические основы и ставится задача, во втором — описываются алгоритм, структура программы, результаты тестирования и экспериментов.

1 АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ И ПОСТАНОВКА ЗАДАЧИ

1.1 Понятие редакционного расстояния

Пусть заданы две последовательности $a = a_1a_2\ldots a_m$ и $b = b_1b_2\ldots b_n$. Редакционным расстоянием называется минимальное число операций редактирования, с помощью которых последовательность a можно превратить в последовательность b. В классической постановке разрешены три операции стоимостью 1: вставка символа, удаление символа и замена одного символа другим [3].

Расстояние названо в честь советского математика В. И. Левенштейна, который в 1965 году исследовал двоичные коды с исправлением выпадений, вставок и замещений символов [1]. При этом элементы последовательностей не обязаны быть буквами: это могут быть кодовые точки Unicode, слова или другие дискретные объекты [3].

Примеры: расстояние между строками «кот» и «скот» равно 1 (достаточно вставить букву «с»), между «дом» и «дым» — также 1 (одна замена), а между «кот» и «код» — 1 (замена последней буквы) [3].

Расстояние Левенштейна является метрикой, то есть для любых строк выполняются свойства неотрицательности, симметричности и неравенства треугольника [3, 4]. Благодаря этому его можно использовать в алгоритмах поиска ближайшего слова и кластеризации строк.

Области применения расстояния Левенштейна [3, 8]:

– проверка орфографии и автокоррекция — поиск слова словаря, ближайшего к введённому;

– поиск и дедупликация — выявление почти одинаковых записей в базе данных;

– биоинформатика — сравнение последовательностей ДНК и белков;

– распознавание речи и текста — оценка числа ошибок распознавания;

– антиплагиат и сравнение документов — оценка различия фрагментов текста.

Существуют модификации метрики: расстояние Дамерау — Левенштейна, в котором дополнительно разрешена перестановка двух соседних символов, и взвешенные редакционные расстояния, где операции имеют разную стоимость [3]. В данной работе рассматривается классический вариант с единичными стоимостями.

1.2 Классический алгоритм Вагнера — Фишера

Обозначим через D(i, j) расстояние Левенштейна между префиксом строки a длиной i и префиксом строки b длиной j. Граничные условия соответствуют преобразованию пустой строки [3]:

$$D(i, 0) = i, \quad D(0, j) = j.$$

Для i > 0 и j > 0 значение вычисляется по рекуррентной формуле:

$$D(i, j) = \min \{ D(i - 1, j) + 1;\ D(i, j - 1) + 1;\ D(i - 1, j - 1) + c(i, j) \},$$

где c(i, j) = 0, если $a_i = b_j$, и c(i, j) = 1 в противном случае. Первый вариант соответствует удалению символа из первой строки, второй — вставке символа, третий — совпадению или замене [2, 3].

Алгоритм последовательно заполняет таблицу размером (m + 1) × (n + 1). Ответ находится в правой нижней клетке D(m, n). Подробное изложение алгоритма и доказательство корректности приведены в учебниках по алгоритмам [6, 7] и в монографии по алгоритмам на строках [17]. Псевдокод алгоритма приведён ниже:

Алгоритм 1.1. Классический алгоритм Вагнера — Фишера

```
для i от 0 до m:  D[i][0] = i
для j от 0 до n:  D[0][j] = j
для i от 1 до m:
  для j от 1 до n:
    c = (a[i] == b[j]) ? 0 : 1
    D[i][j] = min(D[i-1][j] + 1,
                  D[i][j-1] + 1,
                  D[i-1][j-1] + c)
```

```
вернуть D[m][n]
```

Пример заполнения матрицы для строк «кот» и «код» приведён в таблице 1.1. Искомое расстояние равно 1 и стоит в правой нижней клетке.

Таблица 1.1 — Матрица расстояний для строк «кот» и «код»

	ε	**к**	**о**	**т**
ε	0	1	2	3
к	1	0	1	2
о	2	1	0	1
д	3	2	1	1

Временная сложность алгоритма составляет $\Theta(m \cdot n)$, поскольку каждая клетка таблицы вычисляется за постоянное время [1, 2, 3]. При хранении всей таблицы память также составляет $\Theta(m \cdot n)$. Именно последнее обстоятельство ограничивает применение алгоритма к длинным строкам.

1.3 Оптимизация памяти

При вычислении клетки $D(i, j)$ используются только значения $D(i - 1, j)$, $D(i, j - 1)$ и $D(i - 1, j - 1)$, то есть предыдущая и текущая строки таблицы. Следовательно, хранить всю матрицу не нужно: достаточно двух массивов длины $n + 1$ [3, 5]. После заполнения очередной строки массивы меняются ролями, поэтому дополнительная память составляет $O(n)$.

Если в качестве «столбцов» использовать более короткую строку, объём памяти уменьшается до $O(\min(m, n))$ [3]. Такая оптимизация не меняет асимптотику времени, но позволяет обрабатывать строки, для которых полная матрица не помещается в оперативную память.

Платой за экономию является невозможность восстановить редакционное предписание (последовательность операций) без дополнительных приёмов. Один из способов — линейно-пространственный алгоритм Хиршберга, который восстанавливает выравнивание за $O(m \cdot n)$ времени и $O(\min(m, n))$ памяти [13]. В данной работе редакционное предписание не строится, поэтому ограничение несущественно.

Известны и другие способы ускорения. Алгоритм Укконена ограничивает вычисления полосой вокруг главной диагонали и работает за $O(k \cdot \min(m, n))$, где k — максимально допустимое расстояние [11]. Бит-параллельный алгоритм Майерса кодирует состояния динамического программирования битами машинного слова и выполняет $O(n \cdot \lceil m/w \rceil)$ операций, где w — разрядность слова [12]. Существенно более быстрые точные алгоритмы для произвольных строк маловероятны: известны условные нижние оценки, связанные с гипотезой SETH [16].

Сравнение рассмотренных подходов приведено в таблице 1.2.

Таблица 1.2 — Сравнение подходов к вычислению расстояния Левенштейна

Подход	**Время**	**Память**	**Особенности**
Полная матрица	$O(m \cdot n)$	$O(m \cdot n)$	простота, возможность восстановить путь
Две строки	$O(m \cdot n)$	$O(\min(m, n))$	путь не восстанавливается
Алгоритм Хиршберга	$O(m \cdot n)$	$O(\min(m, n))$	восстановление пути, рекурсия
Алгоритм Укконена	$O(k \cdot \min(m, n))$	$O(k \cdot \min(m, n))$	эффективен при малом k
Алгоритм Майерса	$O(n \cdot \lceil m/w \rceil)$	$O(\lceil m/w \rceil)$	битовые операции, $m \leq w$
Параллельная обработка пар	$O(m \cdot n / p)$	$O(p \cdot \min(m, n))$	ускорение на наборе строк

1.4 Способы распараллеливания вычислений

Возможности параллельных вычислений ограничены законом Амдала. Если доля последовательной части программы равна s, то ускорение при использовании p процессоров не превышает величины:

$$S(p) = 1 / (s + (1 - s) / p).$$

Из формулы следует, что даже небольшая последовательная часть существенно ограничивает ускорение при большом числе потоков. Поэтому при распараллеливании важно минимизировать синхронизацию и долю последовательного кода.

Первый способ — распараллеливание по антидиагоналям матрицы. Клетки с одинаковой суммой $i + j$ не зависят друг от друга (рисунок 1.1), поэтому их можно вычислять одновременно, а между диагоналями ставить

барьер [15]. Недостаток способа в том, что число барьеров равно m + n, а на коротких диагоналях полезной работы мало; в результате накладные расходы на синхронизацию могут превысить выигрыш [14].

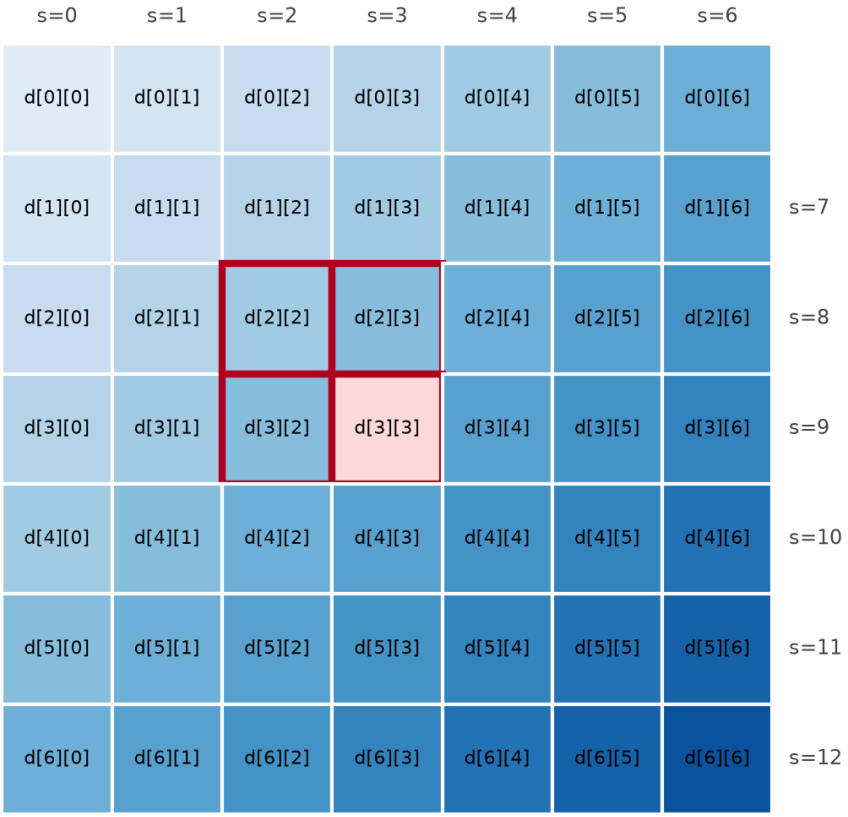


Рисунок 1.1 — Антидиагонали матрицы расстояний (красным выделены клетка d[3][3] и клетки, от которых она зависит)

Второй способ — блочное (wavefront) разбиение матрицы на блоки. Блоки, расположенные на одной блок-диагонали, независимы, поэтому число синхронизаций уменьшается пропорционально размеру блока. Такой подход позволяет лучше загрузить потоки, но требует более сложной реализации и аккуратной передачи граничных значений между блоками [14].

Третий способ — параллельная обработка набора независимых пар строк. В реальных задачах расстояние обычно нужно вычислить не для одной пары, а для многих (сравнение запроса со словарём, поиск дубликатов, пакетная проверка документов). Пары не связаны между собой, поэтому каждая задача может выполняться независимо, а ускорение ограничивается только числом ядер и равномерностью загрузки [15]. Этот способ не

уменьшает время вычисления одной пары, но увеличивает общую производительность программы.

В языке Java для организации параллельных вычислений предназначены средства пакета java.util.concurrent: интерфейс ExecutorService с пулом потоков, интерфейсы Callable и Future, а также пул ForkJoinPool с поддержкой разделяемых задач [9, 10]. Пул потоков создаётся один раз и переиспользует потоки, что снижает накладные расходы по сравнению с созданием нового потока для каждой задачи.

1.5 Постановка задачи на разработку

На основании проведённого анализа формулируется техническое задание.

Назначение программы: вычисление расстояния Левенштейна между строками, а также пакетная обработка набора пар строк с использованием нескольких потоков.

Входные данные: две строки произвольной длины (для матричной версии — до нескольких тысяч символов, для версии с оптимизацией памяти — до сотен тысяч), либо два массива строк одинаковой длины для пакетного режима; число потоков задаётся при создании объекта.

Выходные данные: целое неотрицательное число — расстояние для одной пары; массив целых чисел — расстояния для набора пар; время работы и объём памяти — при выполнении замеров.

Функциональные требования:

– Ф1 — вычисление расстояния для одной пары строк классическим методом;

– Ф2 — вычисление расстояния с хранением только двух строк матрицы;

– Ф3 — параллельное вычисление расстояний для набора пар строк с заданным числом потоков;

– Ф4 — вывод результатов в консоль и сохранение результатов замеров в файл.

Нефункциональные требования:

– Н1 — корректность: результаты должны совпадать с классическим алгоритмом на всех тестовых наборах;

– Н2 — экономия памяти: для версии с оптимизацией память должна составлять O(min(m, n));

– Н3 — кроссплатформенность: программа должна работать на любой платформе, где установлена виртуальная машина Java;

– Н4 — отсутствие внешних библиотек: используется только стандартная библиотека Java;

– Н5 — настраиваемое число потоков.

Ограничения: строки рассматриваются как последовательности символов UTF-16 (тип char), стоимости всех операций равны единице, редакционное предписание не восстанавливается.

1.6 Выводы по разделу 1

В первом разделе рассмотрено понятие расстояния Левенштейна, классический алгоритм Вагнера — Фишера и его сложность O(m·n) по времени и памяти. Показано, что хранение двух строк вместо всей матрицы снижает память до O(min(m, n)) без изменения времени работы. Рассмотрены способы распараллеливания и обоснован выбор пакетной обработки независимых пар строк: она не требует барьеров между диагоналями и обеспечивает предсказуемое ускорение. Сформулировано техническое задание на разработку.

2 РАЗРАБОТКА АЛГОРИТМА И ЕГО ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

2.1 Выбор средств разработки

Программа реализована на языке Java (JDK 21 LTS). Выбор языка обусловлен следующими причинами. Во-первых, Java является кроссплатформенным языком: скомпилированный байт-код выполняется на любой платформе, для которой существует виртуальная машина Java. Во-вторых, стандартная библиотека содержит развитые средства многопоточности — ExecutorService, Callable, Future, ForkJoinPool [9, 10], — что позволяет реализовать параллельные вычисления без сторонних библиотек. В-третьих, автоматическое управление памятью упрощает работу с массивами и исключает ошибки ручного освобождения памяти.

Для сборки используется компилятор javac, входящий в состав JDK. Внешние библиотеки не применяются, поэтому проект собирается одной командой и не требует системы управления зависимостями.

2.2 Структура программы

Все реализации алгоритма подключаются через общий интерфейс DistanceAlgorithm, содержащий метод distance(String a, String b) и метод name(), возвращающий название реализации. Благодаря этому тесты и замеры работают с массивом объектов интерфейса и не зависят от конкретного класса. Состав классов приведён в таблице 2.1.

Таблица 2.1 — Состав классов программы

Класс или интерфейс	Назначение
DistanceAlgorithm	общий интерфейс всех реализаций
LevenshteinMatrix	классический алгоритм с полной матрицей
LevenshteinTwoRows	алгоритм с хранением двух строк
LevenshteinParallel	параллельная обработка набора пар строк
Main	демонстрация работы и расчёт для одной пары
TestRunner	проверка корректности реализаций
Benchmark	замеры времени и сохранение результатов
MemoryProbe	проверка предельного объёма памяти

Класс LevenshteinParallel содержит пул потоков, создаваемый в конструкторе, и реализует интерфейс AutoCloseable, поэтому пул корректно завершается методом close(). Классы LevenshteinMatrix и LevenshteinTwoRows не имеют состояния и могут использоваться повторно.

2.3 Разработка алгоритма

Блок-схема работы программы приведена на рисунке 2.1. Алгоритм состоит из следующих шагов: ввод строк; проверка граничного случая, когда одна из строк пуста; инициализация первой строки и первого столбца матрицы; заполнение клеток по рекуррентной формуле; вывод результата.

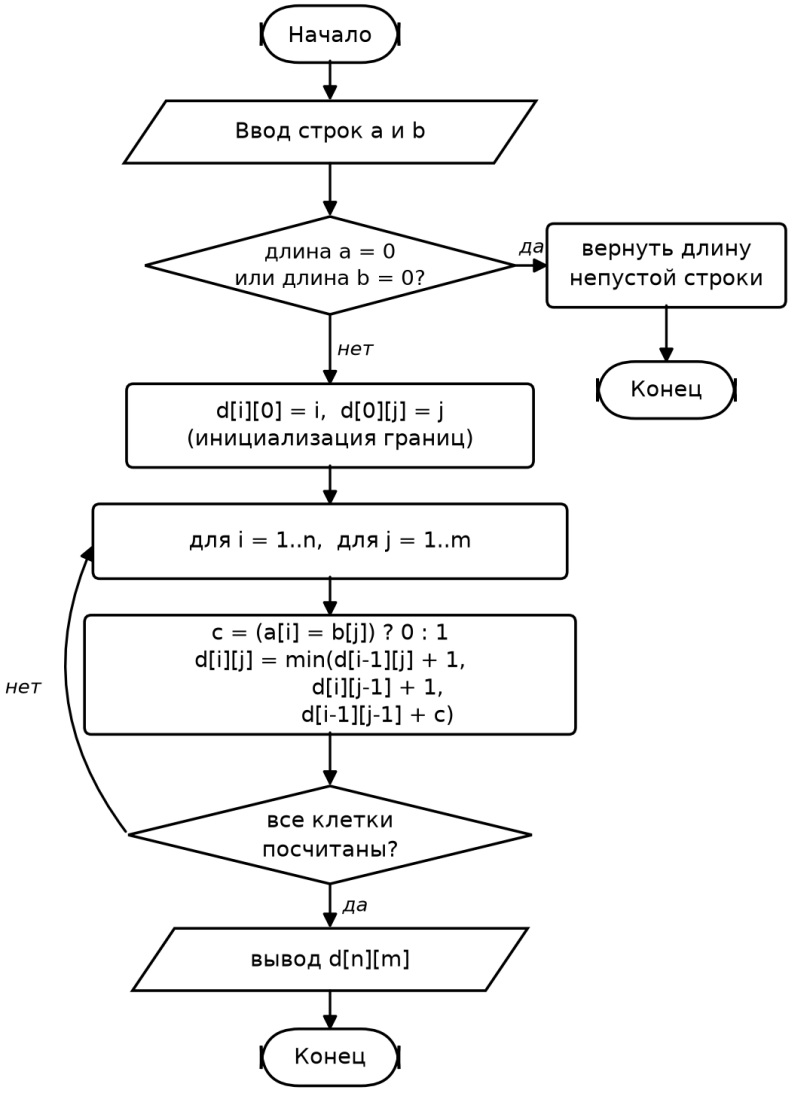


Рисунок 2.1 — Блок-схема алгоритма

Для параллельной версии разработан алгоритм пакетной обработки. Он получает два массива строк одинаковой длины, создаёт для каждой пары отдельную задачу Callable и передаёт все задачи пулу потоков. Пул выполняет задачи параллельно и возвращает массив результатов в исходном порядке. Псевдокод алгоритма приведён ниже.

Алгоритм 2.1. Параллельная обработка набора пар строк

```
функция РасстоянияПараллельно(A, B):
  пул ← Executors.newFixedThreadPool(p)
  задачи ← пустой список
  для i от 0 до |A| - 1:
    задачи.add(() -> ЛевенштейнДвеСтроки(A[i], B[i]))
  результаты ← пул.invokeAll(задачи)
  вернуть [f.get() для f в результаты]
```

Число потоков p задаётся при создании объекта. Если p = 1, задачи выполняются последовательно, что используется как база для расчёта ускорения. Порядок результатов сохраняется, поскольку invokeAll возвращает список Future в том же порядке, в котором были переданы задачи [10].

2.4 Реализация классического алгоритма

Класс LevenshteinMatrix создаёт двумерный массив размером (n + 1) × (m + 1), заполняет граничные значения и вычисляет каждую клетку по рекуррентной формуле. Ключевой фрагмент реализации приведён в листинге 2.1.

Листинг 2.1. Фрагмент класса LevenshteinMatrix

```
int[][] d = new int[n + 1][m + 1];
for (int i = 0; i <= n; i++) d[i][0] = i;
for (int j = 0; j <= m; j++) d[0][j] = j;


for (int i = 1; i <= n; i++) {
  char ca = a.charAt(i - 1);
  for (int j = 1; j <= m; j++) {
    int cost = (ca == b.charAt(j - 1)) ? 0 : 1;
    int deletion = d[i - 1][j] + 1;
    int insertion = d[i][j - 1] + 1;
    int substitution = d[i - 1][j - 1] + cost;
    d[i][j] = Math.min(Math.min(deletion, insertion), substitution);
```

```
  }
}
return d[n][m];
```

Класс используется как эталон: его результат считается правильным при проверке остальных реализаций. Временная сложность — O(n·m), память — O(n·m).

2.5 Реализация с оптимизацией памяти

В классе LevenshteinTwoRows хранятся два массива prev и cur длины m + 1. После заполнения очередной строки массивы меняются местами, поэтому копирование не требуется. Если первая строка короче второй, строки меняются местами: расстояние Левенштейна симметрично, а в качестве «столбцов» выгодно использовать более короткую строку, что даёт память O(min(n, m)). Фрагмент реализации приведён в листинге 2.2.

Листинг 2.2. Фрагмент класса LevenshteinTwoRows

```
if (a.length() < b.length()) {   // короткая строка - столбцы
   String tmp = a; a = b; b = tmp;
}
int[] prev = new int[m + 1];
int[] cur  = new int[m + 1];
for (int j = 0; j <= m; j++) prev[j] = j;


for (int i = 1; i <= n; i++) {
   cur[0] = i;
   char ca = a.charAt(i - 1);
   for (int j = 1; j <= m; j++) {
      int cost = (ca == b.charAt(j - 1)) ? 0 : 1;
      int deletion = prev[j] + 1;
      int insertion = cur[j - 1] + 1;
      int substitution = prev[j - 1] + cost;
      cur[j] = Math.min(Math.min(deletion, insertion), substitution);
   }
   int[] tmp = prev; prev = cur; cur = tmp;   // обмен строк
}
return prev[m];
```

Обмен ссылками вместо копирования массива важен для производительности: копирование строки длины m на каждом из n шагов

дало бы дополнительно O(n·m) операций. В результате время работы остаётся O(n·m), а память сокращается до O(min(n, m)).

2.6 Реализация параллельных вычислений

Класс LevenshteinParallel реализует пакетную обработку. В конструкторе создаётся пул фиксированного размера; метод distances принимает два массива строк, формирует список задач Callable и выполняет их методом invokeAll, который возвращает управление после завершения всех задач [10]. Каждая задача вычисляет расстояние последовательным алгоритмом с оптимизацией памяти. Фрагмент реализации приведён в листинге 2.3.

Листинг 2.3. Фрагмент класса LevenshteinParallel

```
List<Callable<Integer>> tasks = new ArrayList<>();
for (int i = 0; i < a.length; i++) {
   final int index = i;
   tasks.add(() -> sequential.distance(a[index], b[index]));
}

List<Future<Integer>> futures = pool.invokeAll(tasks);
int[] result = new int[a.length];
for (int i = 0; i < result.length; i++) {
   result[i] = futures.get(i).get();
}
return result;
```

Выбор пакетной схемы объясняется результатами раздела 1.4: распараллеливание внутри одной пары строк требует барьера на каждой антидиагонали, и накладные расходы на синхронизацию растут пропорционально длине строк [14]. Пакетная схема, напротив, не содержит синхронизации между задачами: потоки только получают задачи из очереди пула. Поэтому она показывает устойчивое ускорение, а память остаётся линейной по длине строки и числу потоков.

2.7 Тестирование программы

Тестирование проводилось в два этапа. На первом этапе использовались фиксированные примеры с заранее известным ответом, включая граничные случаи пустых строк и полностью совпадающих строк. Результаты приведены в таблице 2.2.

Таблица 2.2 — Результаты проверки на фиксированных примерах

Строка A	Строка B	Ожидается	Матрица	Две строки	Параллельно
кот	код	1	1	1	1
кот	скот	1	1	1	1
дом	дым	1	1	1	1
hello	yellow	2	2	2	2
		0	0	0	0
	abc	3	3	3	3
abc		3	3	3	3
abc	abc	0	0	0	0
abcdef	azced	3	3	3	3
расстояние	рост	7	7	7	7

На втором этапе выполнялось сравнение с эталонной матричной реализацией на случайных строках: 300 пар длиной от 0 до 200 символов над алфавитом из 30 букв, а также 40 пар длиной от 200 до 400 символов в пакетном режиме. Во всех случаях результаты совпали с эталоном, расхождений не обнаружено. Дополнительно проверялись строки одинаковой длины, одна пустая строка, полностью различные строки и строки с повторяющимися символами.

Тестирование показало, что все три реализации корректны и могут использоваться для дальнейших экспериментов.

2.8 Результаты экспериментальных замеров

Замеры выполнялись на ноутбуке с процессором AMD Ryzen 5 PRO 3500U (4 физических ядра, 8 логических потоков, базовая частота 2,1 ГГц) под управлением Windows. Использовалась Java 21 с ограничением кучи -Xmx1g. Перед основными замерами выполнялся прогрев: 30 итераций для каждой реализации, чтобы JIT-компилятор оптимизировал код. Время

измерялось методом System.nanoTime(), для каждой конфигурации выполнялось 7 запусков и выбиралось минимальное значение, что снижает влияние случайных задержек операционной системы.

Результаты для одной пары строк приведены в таблице 2.3 и на рисунке 2.2.

Таблица 2.3 — Время вычисления расстояния для одной пары строк, мс

Длина строк n = m	Полная матрица	Две строки
100	0,151	0,094
250	0,933	0,320
500	2,028	1,179
1000	8,632	4,486
2000	31,693	28,079
4000	201,803	92,062

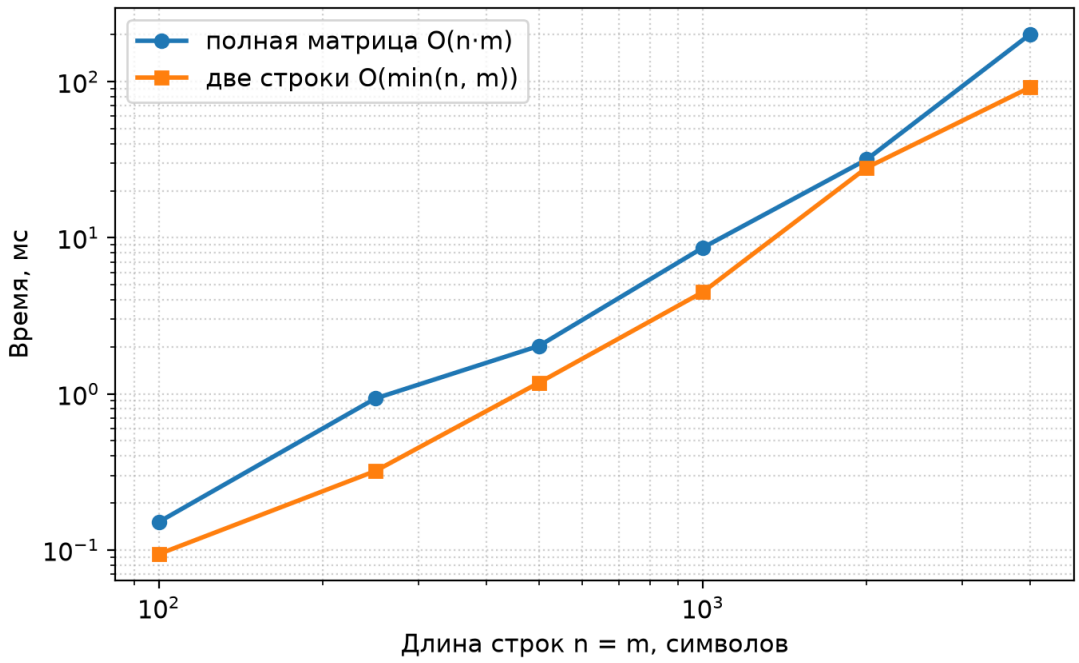


Рисунок 2.2 — Зависимость времени работы от длины строк

Версия с двумя строками быстрее матричной при всех длинах, а разрыв увеличивается с ростом n: при n = 4000 время отличается более чем в два раза. Это объясняется не только объёмом памяти, но и лучшей локальностью: два массива длины n + 1 помещаются в кэш процессора, тогда как матрица размером 64 МБ при n = 4000 постоянно вытесняется в оперативную память.

Результаты пакетной параллельной обработки приведены в таблице 2.4 и на рисунке 2.3. Обрабатывался набор из 48 пар строк одинаковой длины; ускорение рассчитано относительно одного потока.

Таблица 2.4 — Время обработки 48 пар строк и ускорение, мс

Длина	1 поток	2 потока	4 потока	8 потоков	Ускорение (8)
500	95,2	39,9	22,2	15,1	6,29
1000	320,2	179,7	99,1	61,5	5,20
2000	861,1	539,2	325,9	225,0	3,83

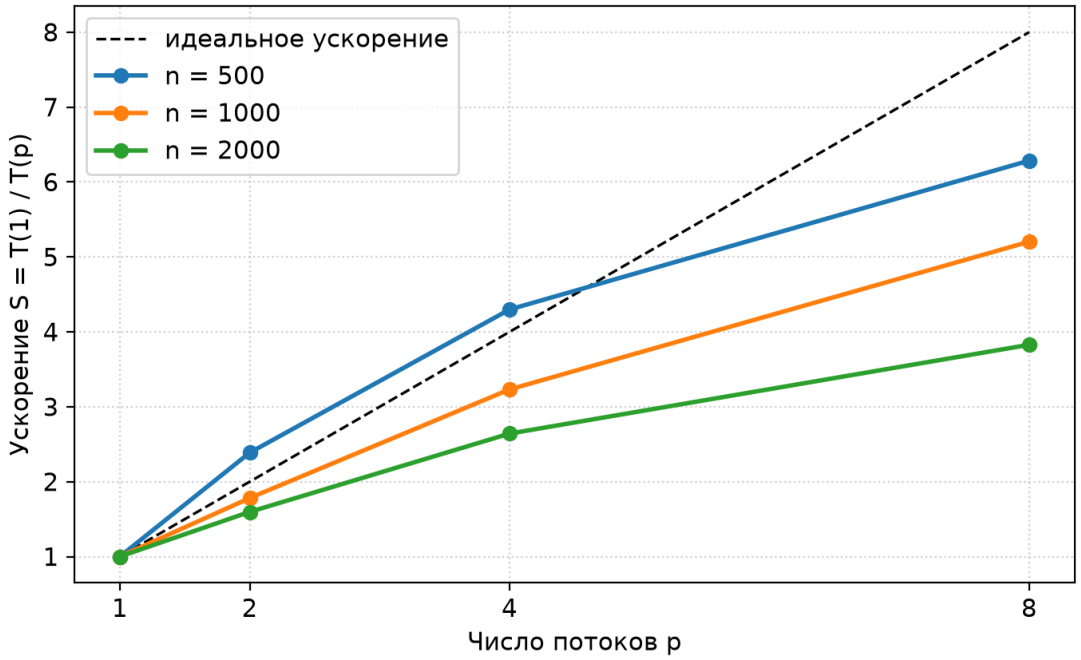


Рисунок 2.3 — Ускорение от числа потоков

Наибольшее ускорение достигается на строках средней длины: при n = 500 оно составляет 6,29, при n = 1000 — 5,20, при n = 2000 — 3,83. Снижение ускорения с ростом n объясняется тем, что при больших объёмах данных программа становится ограничена пропускной способностью памяти, а не вычислениями, и добавление потоков уже не даёт пропорционального прироста. Кроме того, на 4 физических ядрах восемь потоков конкурируют за исполнительные устройства (технология SMT даёт лишь частичный прирост).

Теоретический объём памяти приведён в таблице 2.5 и на рисунке 2.4. Значения рассчитаны по формулам $(n + 1)^2 \cdot 4$ байта для полной матрицы и $2 \cdot (n + 1) \cdot 4$ байта для двух строк.

Таблица 2.5 — Теоретический объём памяти, байт

Длина	Полная матрица	Две строки
100	40 804	808
250	252 004	2 008
500	1 004 004	4 008
1000	4 008 004	8 008
2000	16 016 004	16 008
4000	64 032 004	32 008

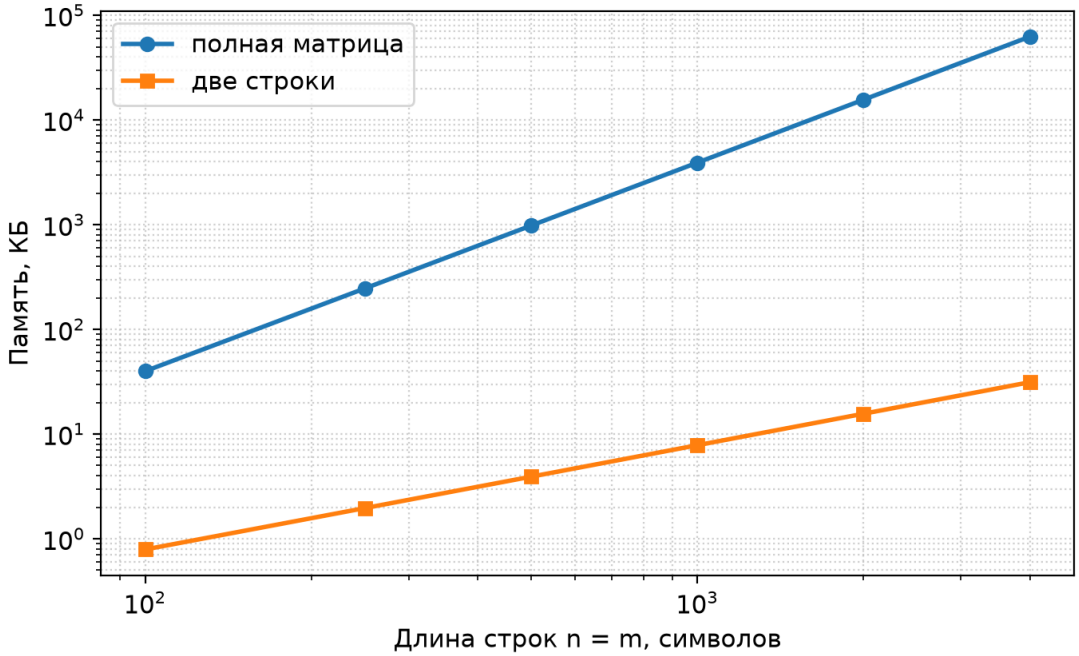


Рисунок 2.4 — Зависимость объёма памяти от длины строк

Разница достигает трёх порядков: при n = 4000 матричная версия требует около 61 МБ, а версия с двумя строками — 32 КБ. Для проверки на практике использовалась программа MemoryProbe, запускаемая с ограничением кучи 256 МБ. Матричная версия успешно выделяет память при n = 8000 (244 МБ) и завершается с ошибкой OutOfMemoryError при n = 9000, тогда как версия с двумя строками выделяет массивы для n = 1 000 000 (около 7,6 МБ) без затруднений.

2.9 Выводы по разделу 2

Разработана программа на языке Java, содержащая три реализации алгоритма Левенштейна: классическую матричную, версию с двумя строками и параллельную пакетную. Проведено тестирование на фиксированных примерах и случайных строках — расхождений с эталоном нет. Эксперименты подтвердили, что оптимизация памяти снижает её объём с O(n·m) до O(min(n, m)) (при n = 4000 — с 61 МБ до 32 КБ) и уменьшает время работы за счёт лучшей локальности данных. Параллельная обработка набора пар даёт ускорение до 6,29 на 8 потоках, однако с ростом длины строк ускорение снижается до 3,83, что объясняется ограничением пропускной способности памяти и числом физических ядер.

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы была достигнута поставленная цель: разработан и программно реализован алгоритм вычисления расстояния Левенштейна с оптимизацией памяти и параллельным вычислением.

В теоретической части рассмотрены понятие редакционного расстояния, классический алгоритм Вагнера — Фишера, его временная и пространственная сложность, а также способы оптимизации памяти и распараллеливания. Обоснован выбор хранения двух строк матрицы и пакетной обработки независимых пар строк.

В практической части на языке Java реализованы три версии алгоритма, объединённые общим интерфейсом, а также программа тестирования и программа замеров. Корректность подтверждена сравнением с эталонной матричной реализацией на фиксированных примерах и на 340 случайных парах строк.

Эксперименты показали, что оптимизация памяти сокращает её объём с O(n·m) до O(min(n, m)): при длине строк 4000 символов — с 61 МБ до 32 КБ, а при ограничении кучи 256 МБ матричная версия работает до n = 8000, тогда как версия с двумя строками — при n = 1 000 000. Параллельная обработка набора пар строк даёт ускорение до 6,29 на 8 потоках, при этом с ростом длины строк ускорение снижается до 3,83 из-за ограничений пропускной способности памяти.

Практическая значимость работы состоит в том, что полученные реализации можно использовать в задачах проверки орфографии, поиска дубликатов, сравнения результатов распознавания и других задачах, где требуется обрабатывать большое число пар строк при ограниченном объёме памяти. Программа не требует внешних библиотек и работает на любой платформе с виртуальной машиной Java.

Направления дальнейшего развития: распараллеливание внутри одной пары строк по антидиагоналям или блокам, реализация бит-параллельного

алгоритма Майерса, восстановление редакционного предписания методом Хиршберга, использование ForkJoinPool для динамической балансировки и применение векторных инструкций процессора.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Левенштейн В. И. Двоичные коды с исправлением выпадений, вставок и замещений символов // Доклады Академии наук СССР. — 1965. — Т. 163, № 4. — С. 845–848.

2. Wagner R. A., Fischer M. J. The String-to-String Correction Problem // Journal of the ACM. — 1974. — Vol. 21, no. 1. — P. 168–173.

3. Расстояние Левенштейна // Википедия : свободная энциклопедия. — URL: https://ru.wikipedia.org/wiki/Расстояние_Левенштейна (дата обращения: 21.10.2026).

4. Levenshtein distance // Wikipedia : the free encyclopedia. — URL: https://en.wikipedia.org/wiki/Levenshtein_distance (date of access: 21.10.2026).

5. Расстояние Левенштейна // e-maxx : алгоритмы и структуры данных. — URL: https://e-maxx.ru/algo/levenshtein_distance (дата обращения: 21.10.2026).

6. Кормен Т. Х., Лейзерсон Ч. И., Ривест Р. Л., Штайн К. Алгоритмы: построение и анализ. — 3-е изд. — М. : Вильямс, 2013. — 1328 с.

7. Седжвик Р. Алгоритмы на Java. — 4-е изд. — М. : Вильямс, 2013. — 848 с.

8. How to Calculate Levenshtein Distance in Java? // Baeldung. — URL: https://www.baeldung.com/java-levenshtein-distance (date of access: 21.10.2026).

9. Class ForkJoinPool // Oracle. Java Platform, Standard Edition 21 API Specification. — URL: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/ForkJoinPool.html (date of access: 21.10.2026).

10. Interface ExecutorService // Oracle. Java Platform, Standard Edition 21 API Specification. — URL: https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/util/concurrent/ExecutorService.html (date of access: 21.10.2026).

11. Ukkonen E. Algorithms for approximate string matching // Information and Control. — 1985. — Vol. 64, no. 1–3. — P. 100–118.

12. Myers G. A fast bit-vector algorithm for approximate string matching based on dynamic programming // Journal of the ACM. — 1999. — Vol. 46, no. 3. — P. 395–415.

13. Hirschberg D. S. A linear space algorithm for computing maximal common subsequences // Communications of the ACM. — 1975. — Vol. 18, no. 6. — P. 341–343.

14. Ding X., Dong X., Gu Y., Liu Y., Sun Y. Efficient Parallel Output-Sensitive Edit Distance // 31st Annual European Symposium on Algorithms (ESA 2023). — 2023. — arXiv:2306.17461.

15. Wright G. Improved Time Warp Edit Distance — A Parallel Dynamic Program in Linear Memory. — 2020. — arXiv:2007.16135.

16. Backurs A., Indyk P. Edit Distance Cannot Be Computed in Strongly Subquadratic Time (unless SETH is false) // Proceedings of the 47th Annual ACM Symposium on Theory of Computing. — 2015. — P. 51–58.

17. Гасфилд Д. Строки, деревья и последовательности в алгоритмах. Информатика и вычислительная биология. — СПб. : Невский Диалект, 2003. — 654 с.

ПРИЛОЖЕНИЕ А

Полный текст программы

Ниже приведён исходный код всех классов программы. Файлы расположены в папке Программа/src/levenshtein.

A.1. Интерфейс DistanceAlgorithm

```
package levenshtein;


/**
* Общий интерфейс для всех реализаций алгоритма вычисления
* расстояния Левенштейна. Позволяет сравнивать реализации
* между собой в тестах и бенчмарках.
*/
public interface DistanceAlgorithm {


   /**
    * Вычисляет расстояние Левенштейна между строками a и b.
    *
    * @param a первая строка
    * @param b вторая строка
    * @return минимальное число вставок, удалений и замен
    */
   int distance(String a, String b);


   /**
    * @return человекочитаемое название реализации
    */
   String name();
}
```

A.2. Класс LevenshteinMatrix

```
package levenshtein;


/**
* Классическая реализация алгоритма Вагнера — Фишера.
* Строит полную матрицу расстояний размером (n + 1) x (m + 1).
*
* Время:  O(n * m)
* Память: O(n * m)
*
```

```
 * Используется как эталон корректности для остальных реализаций.
 */
public final class LevenshteinMatrix implements DistanceAlgorithm {

  @Override
  public int distance(String a, String b) {
    int n = a.length();
    int m = b.length();

    int[][] d = new int[n + 1][m + 1];

    // Граничные условия: расстояние от пустой строки.
    for (int i = 0; i <= n; i++) {
      d[i][0] = i;
    }
    for (int j = 0; j <= m; j++) {
      d[0][j] = j;
    }

    // Основной цикл динамического программирования.
    for (int i = 1; i <= n; i++) {
      char ca = a.charAt(i - 1);
      for (int j = 1; j <= m; j++) {
        int cost = (ca == b.charAt(j - 1)) ? 0 : 1;

        int deletion = d[i - 1][j] + 1;
        int insertion = d[i][j - 1] + 1;
        int substitution = d[i - 1][j - 1] + cost;

        d[i][j] = Math.min(Math.min(deletion, insertion), substitution);
      }
    }

    return d[n][m];
  }

  @Override
  public String name() {
    return "Полная матрица";
  }
}
```

A.3. Класс LevenshteinTwoRows

```
package levenshtein;
```

```
/**
 * Реализация с оптимизацией памяти.
 *
 * Для вычисления очередной строки матрицы достаточно знать
 * только предыдущую строку, поэтому хранятся два массива длины m + 1.
 * Более короткая строка всегда используется как "столбцы",
 * поэтому памяти нужно O(min(n, m)).
 *
 * Время:  O(n * m)
 * Память: O(min(n, m))
 */
public final class LevenshteinTwoRows implements DistanceAlgorithm {

  @Override
  public int distance(String a, String b) {
    // Для экономии памяти внешним циклом идём по более длинной строке.
    if (a.length() < b.length()) {
      String tmp = a;
      a = b;
      b = tmp;
    }

    int n = a.length();
    int m = b.length();

    int[] prev = new int[m + 1];
    int[] cur = new int[m + 1];

    for (int j = 0; j <= m; j++) {
      prev[j] = j;
    }

    for (int i = 1; i <= n; i++) {
      cur[0] = i;
      char ca = a.charAt(i - 1);

      for (int j = 1; j <= m; j++) {
        int cost = (ca == b.charAt(j - 1)) ? 0 : 1;

        int deletion = prev[j] + 1;
        int insertion = cur[j - 1] + 1;
        int substitution = prev[j - 1] + cost;

        cur[j] = Math.min(Math.min(deletion, insertion), substitution);
      }

      // Меняем строки местами, чтобы не копировать массивы.
```

```
            int[] tmp = prev;
            prev = cur;
            cur = tmp;
        }

        return prev[m];
    }

    @Override
    public String name() {
        return "Две строки O(min(n,m))";
    }
}
```

A.4. Класс LevenshteinParallel

```
package levenshtein;


import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutionException;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
import java.util.concurrent.TimeUnit;


/**
 * Параллельная версия.
 *
 * В реальных задачах расстояние Левенштейна обычно нужно вычислить
 * не для одной пары строк, а для большого набора пар (поиск дубликатов,
 * проверка орфографии, сравнение результатов распознавания и т.п.).
 * Пары строк независимы друг от друга, поэтому их обработку можно
 * распределить между потоками: каждая задача считает расстояние
 * обычным последовательным алгоритмом с оптимизацией памяти.
 *
 * Внутри одной пары строк распараллеливание затруднено: клетки матрицы
 * зависят друг от друга по диагоналям, и на каждую диагональ пришлось бы
 * ставить барьер (см. раздел 1.4 пояснительной записки).
 *
 * Память: O(p * min(n, m)), где p - число потоков.
 */
public final class LevenshteinParallel implements DistanceAlgorithm, AutoCloseable {


    private final int threads;
    private final ExecutorService pool;
```

```
private final LevenshteinTwoRows sequential = new LevenshteinTwoRows();


public LevenshteinParallel(int threads) {
  if (threads < 1) {
    throw new IllegalArgumentException("Число потоков должно быть >= 1");
  }
  this.threads = threads;
  this.pool = Executors.newFixedThreadPool(threads);
}


/** Расстояние для одной пары строк (выполняется последовательно). */
@Override
public int distance(String a, String b) {
  return sequential.distance(a, b);
}


/**
 * Вычисляет расстояния для набора пар строк параллельно.
 *
 * @param a массив первых строк
 * @param b массив вторых строк
 * @return массив расстояний, result[i] = distance(a[i], b[i])
 */
public int[] distances(final String[] a, final String[] b) {
  if (a.length != b.length) {
    throw new IllegalArgumentException("Массивы строк должны быть одинаковой длины");
  }


  List<Callable<Integer>> tasks = new ArrayList<Callable<Integer>>(a.length);
  for (int i = 0; i < a.length; i++) {
    final int index = i;
    tasks.add(new Callable<Integer>() {
      @Override
      public Integer call() {
        return sequential.distance(a[index], b[index]);
      }
    });
  }


  try {
    List<Future<Integer>> futures = pool.invokeAll(tasks);
    int[] result = new int[a.length];
    for (int i = 0; i < result.length; i++) {
      result[i] = futures.get(i).get();
    }
    return result;
  } catch (InterruptedException e) {
    Thread.currentThread().interrupt();
```

```
            throw new IllegalStateException("Вычисление прервано", e);
        } catch (ExecutionException e) {
            throw new IllegalStateException("Ошибка при вычислении", e.getCause());
        }
    }

    public int getThreads() {
        return threads;
    }

    @Override
    public String name() {
        return "Параллельная (" + threads + " потоков)";
    }

    @Override
    public void close() {
        pool.shutdown();
        try {
            if (!pool.awaitTermination(5, TimeUnit.SECONDS)) {
                pool.shutdownNow();
            }
        } catch (InterruptedException e) {
            pool.shutdownNow();
            Thread.currentThread().interrupt();
        }
    }
}
```

A.5. Класс Main

```
package levenshtein;


/**
* Мини-программа для демонстрации работы алгоритма
* и проверки на нескольких характерных примерах.
*
* Запуск без аргументов:  java -cp out levenshtein.Main
* Запуск с аргументами:   java -cp out levenshtein.Main "кот" "код"
*/
public final class Main {

    public static void main(String[] args) {
        if (args.length == 2) {
            DistanceAlgorithm algorithm = new LevenshteinTwoRows();
            int result = algorithm.distance(args[0], args[1]);
            System.out.println("Расстояние Левенштейна: " + result);
```

```
            return;
        }

        System.out.println("Демонстрация работы алгоритма Левенштейна");
        System.out.println("----------------------------------------");

        DistanceAlgorithm matrix = new LevenshteinMatrix();
        DistanceAlgorithm twoRows = new LevenshteinTwoRows();

        String[][] examples = {
                {"кот", "код"},
                {"кот", "скот"},
                {"дом", "дым"},
                {"hello", "yellow"},
                {"", "abc"},
                {"abc", "abc"}
        };

        for (String[] pair : examples) {
            int a = matrix.distance(pair[0], pair[1]);
            int b = twoRows.distance(pair[0], pair[1]);
            System.out.printf("%-10s -> %-10s : %d (матрица) / %d (две строки)%n",
                    "\"" + pair[0] + "\"", "\"" + pair[1] + "\"", a, b);
        }

        System.out.println();
        System.out.println("Для проверки всех реализаций запустите:");
        System.out.println("  java -cp out levenshtein.TestRunner");
        System.out.println("Для замеров времени запустите:");
        System.out.println("  java -Xmx1g -cp out levenshtein.Benchmark");
    }
}
```

A.6. Класс TestRunner

```
package levenshtein;

import java.util.Random;

/**
 * Проверка корректности всех реализаций.
 *
 * 1. Фиксированные примеры с заранее известным ответом.
 * 2. Случайные пары строк: результат каждой реализации сравнивается
 *    с эталонной матричной версией.
 * 3. Пакетный параллельный режим: набор пар обрабатывается пулом потоков.
 *
```

```
 * Запуск: java -cp out levenshtein.TestRunner
 */
public final class TestRunner {

  private static final String ALPHABET = "абвгдежзийклмнопрстуфхцчшщыэюя";

  public static void main(String[] args) {
    int failed = 0;

    failed += checkFixedExamples();
    failed += checkRandomExamples(300, 200);
    failed += checkBatchMode(40, 200, 400);

    System.out.println();
    if (failed == 0) {
      System.out.println("ИТОГ: все тесты пройдены успешно.");
    } else {
      System.out.println("ИТОГ: обнаружено ошибок: " + failed);
      System.exit(1);
    }
  }

  /** Проверка на примерах, ответ для которых известен заранее. */
  private static int checkFixedExamples() {
    System.out.println("Таблица 1. Проверка на фиксированных примерах");
    System.out.println("-----------------------------------------------------------------");
    System.out.printf("%-20s %-20s %-10s %-10s %-10s %-10s%n",
        "Строка A", "Строка B", "Ожидается", "Матрица", "2 строки", "Параллельно");
    System.out.println("-----------------------------------------------------------------");

    String[][] cases = {
        {"кот", "код", "1"},
        {"кот", "скот", "1"},
        {"дом", "дым", "1"},
        {"hello", "yellow", "2"},
        {"", "", "0"},
        {"", "abc", "3"},
        {"abc", "", "3"},
        {"abc", "abc", "0"},
        {"abcdef", "azced", "3"},
        {"расстояние", "рост", "7"}
    };

    int failed = 0;
    LevenshteinParallel parallel = new LevenshteinParallel(4);
    DistanceAlgorithm[] algorithms = {
        new LevenshteinMatrix(),
```

```
            new LevenshteinTwoRows(),
            parallel
    };
    String[] names = {"Матрица", "2 строки", "Параллельно"};

    for (String[] c : cases) {
        int expected = Integer.parseInt(c[2]);
        System.out.printf("%-20s %-20s %-10d", quote(c[0]), quote(c[1]), expected);

        for (int k = 0; k < algorithms.length; k++) {
            int actual = algorithms[k].distance(c[0], c[1]);
            System.out.printf(" %-10d", actual);
            if (actual != expected) {
                System.out.print(" <-- ОШИБКА (" + names[k] + ")");
                failed++;
            }
        }
        System.out.println();
    }
    parallel.close();
    return failed;
}

/** Сравнение всех реализаций с эталоном на случайных строках. */
private static int checkRandomExamples(int pairs, int maxLength) {
    Random random = new Random(42);
    int failed = 0;

    LevenshteinMatrix reference = new LevenshteinMatrix();
    LevenshteinTwoRows twoRows = new LevenshteinTwoRows();
    LevenshteinParallel parallel = new LevenshteinParallel(4);

    for (int t = 0; t < pairs; t++) {
        String a = randomString(random, random.nextInt(maxLength + 1));
        String b = randomString(random, random.nextInt(maxLength + 1));

        int expected = reference.distance(a, b);
        int actualTwoRows = twoRows.distance(a, b);
        int actualParallel = parallel.distance(a, b);

        if (expected != actualTwoRows || expected != actualParallel) {
            failed++;
            System.out.println("Несовпадение: \"" + a + "\" / \"" + b + "\""
                    + " эталон=" + expected
                    + " две строки=" + actualTwoRows
                    + " параллельно=" + actualParallel);
        }
```

```
    }

    System.out.println();
    System.out.println("Таблица 2. Случайные строки: " + pairs + " пар, "
            + "длина 0..." + maxLength + ", алфавит " + ALPHABET.length() + " букв");
    System.out.println("Несовпадений с эталоном: " + failed);

    parallel.close();
    return failed;
}

/** Проверка пакетного режима: пул потоков считает набор пар. */
private static int checkBatchMode(int pairs, int minLength, int maxLength) {
    Random random = new Random(7);
    LevenshteinMatrix reference = new LevenshteinMatrix();

    String[] as = new String[pairs];
    String[] bs = new String[pairs];
    int[] expected = new int[pairs];

    for (int i = 0; i < pairs; i++) {
        int length = minLength + random.nextInt(maxLength - minLength + 1);
        as[i] = randomString(random, length);
        bs[i] = randomString(random, length);
        expected[i] = reference.distance(as[i], bs[i]);
    }

    int failed = 0;
    try (LevenshteinParallel parallel = new LevenshteinParallel(4)) {
        int[] actual = parallel.distances(as, bs);
        for (int i = 0; i < pairs; i++) {
            if (actual[i] != expected[i]) {
                failed++;
                System.out.println("Несовпадение в пакетном режиме, пара " + i
                        + ": эталон=" + expected[i] + ", получено=" + actual[i]);
            }
        }
    }

    System.out.println();
    System.out.println("Таблица 3. Пакетный параллельный режим: " + pairs
            + " пар, длина " + minLength + "..." + maxLength);
    System.out.println("Несовпадений с эталоном: " + failed);

    return failed;
}
```

```
    private static String randomString(Random random, int length) {
        StringBuilder sb = new StringBuilder(length);
        for (int i = 0; i < length; i++) {
            sb.append(ALPHABET.charAt(random.nextInt(ALPHABET.length())));
        }
        return sb.toString();
    }

    private static String quote(String s) {
        return "\"" + s + "\"";
    }
}
```

A.7. Класс MemoryProbe

```
package levenshtein;


/**
 * Проверка того, сколько памяти требует каждая реализация.
 *
 * Программа запускается с ограничением кучи, например:
 *   java -Xmx256m -cp out levenshtein.MemoryProbe matrix 8000
 *
 * Если памяти хватает, печатается "OK", иначе "OutOfMemoryError".
 * Это позволяет найти максимальную длину строк, при которой
 * реализация ещё работает при заданном объёме памяти.
 */
public final class MemoryProbe {


    public static void main(String[] args) {
        if (args.length != 2) {
            System.out.println("Использование: java -Xmx256m -cp out levenshtein.MemoryProbe <matrix|rows> <n>");
            return;
        }

        String mode = args[0];
        int n = Integer.parseInt(args[1]);


        try {
            if ("matrix".equals(mode)) {
                // Полная матрица (n + 1) x (n + 1) 32-битных чисел.
                int[][] matrix = new int[n + 1][n + 1];
                matrix[n][n] = 1; // обращение, чтобы массив не удалили
                System.out.println("OK: матрица " + (n + 1) + "x" + (n + 1) + " выделена");
                System.out.println("Теоретический объём: "
                        + ((long) (n + 1) * (n + 1) * Integer.BYTES / 1024 / 1024) + " МБ");
            } else if ("rows".equals(mode)) {
```

```
            // Два массива длины n + 1.
            int[] prev = new int[n + 1];
            int[] cur = new int[n + 1];
            prev[n] = 1;
            cur[n] = 1;
            System.out.println("OK: два массива длины " + (n + 1) + " выделены");
            System.out.println("Теоретический объём: "
                    + (2L * (n + 1) * Integer.BYTES / 1024) + " КБ");
        } else {
            System.out.println("Неизвестный режим: " + mode);
        }
    } catch (OutOfMemoryError e) {
        System.out.println("OutOfMemoryError при n = " + n + " (режим " + mode + ")");
    }
  }
}
```

A.8. Класс Benchmark

```
package levenshtein;


import java.io.PrintWriter;
import java.util.Locale;
import java.util.Random;


/**
* Замеры времени работы реализаций.
*
* Таблица 1 - время вычисления расстояния для одной пары строк.
* Таблица 2 - время обработки набора из 32 пар строк в 1, 2, 4 и 8 потоков
*          и ускорение относительно одного потока.
* Таблица 3 - теоретический объём памяти.
*
* Результаты печатаются в консоль и сохраняются в results.md / results.csv.
*
* Запуск: java -Xmx1g -cp out levenshtein.Benchmark
*/
public final class Benchmark {


  private static final int[] SIZES = {100, 250, 500, 1000, 2000, 4000};
  private static final int[] BATCH_SIZES = {500, 1000, 2000};
  private static final int[] THREADS = {1, 2, 4, 8};
  private static final int PAIRS = 48;


  private static final int WARMUP = 2;
  private static final int RUNS = 7;
  private static final int BATCH_WARMUP = 1;
  private static final int BATCH_RUNS = 5;
```

```
private static final String ALPHABET = "abcdefghijklmnopqrstuvwxyz";

/** Защита от удаления "ненужных" вычислений оптимизатором JIT. */
private static volatile long sink;

public static void main(String[] args) throws Exception {
  Locale.setDefault(Locale.US);
  Random random = new Random(42);

  warmUp(random);

  StringBuilder md = new StringBuilder();
  StringBuilder csv = new StringBuilder();

  md.append("# Результаты замеров\n\n");
  md.append("Время в миллисекундах: минимум из ").append(RUNS)
   .append(" запусков после ").append(WARMUP).append(" прогревочных.\n\n");

  md.append("## Таблица 1. Одна пара строк\n\n");
  md.append("| Длина | Полная матрица | Две строки |\n");
  md.append("|---:|---:|---:|\n");

  csv.append("n,matrix,twoRows\n");

  System.out.println("Таблица 1. Одна пара строк, время (мс)");
  System.out.printf("%6s %18s %14s%n", "n", "Полная матрица", "Две строки");

  for (int n : SIZES) {
    String a = randomString(random, n);
    String b = randomString(random, n);

    double matrix = measure(new LevenshteinMatrix(), a, b);
    double twoRows = measure(new LevenshteinTwoRows(), a, b);

    System.out.printf("%6d %18.3f %14.3f%n", n, matrix, twoRows);
    md.append(String.format(Locale.US, "| %d | %.3f | %.3f |%n", n, matrix, twoRows));
    csv.append(String.format(Locale.US, "%d,%.3f,%.3f%n", n, matrix, twoRows));
  }

  md.append("\n## Таблица 2. Набор из ").append(PAIRS).append(" пар строк\n\n");
  md.append("| Длина | 1 поток | 2 потока | 4 потока | 8 потоков | Ускорение (8 потоков) |\n");
  md.append("|---:|---:|---:|---:|---:|---:|\n");

  csv.append("\nbatch_n,p1,p2,p4,p8\n");
```

```
System.out.println();
System.out.println("Таблица 2. Набор из " + PAIRS + " пар строк, время (мс)");
System.out.printf("%6s %10s %10s %10s %10s %20s%n",
      "n", "1 поток", "2 потока", "4 потока", "8 потоков", "Ускорение (8)");

for (int n : BATCH_SIZES) {
   String[] as = new String[PAIRS];
   String[] bs = new String[PAIRS];
   for (int i = 0; i < PAIRS; i++) {
      as[i] = randomString(random, n);
      bs[i] = randomString(random, n);
   }

   double[] time = new double[THREADS.length];
   for (int t = 0; t < THREADS.length; t++) {
      try (LevenshteinParallel parallel = new LevenshteinParallel(THREADS[t])) {
         time[t] = measureBatch(parallel, as, bs);
      }
   }

   double speedup = time[0] / time[time.length - 1];

   System.out.printf("%6d %10.3f %10.3f %10.3f %10.3f %20.2f%n",
         n, time[0], time[1], time[2], time[3], speedup);

   md.append(String.format(Locale.US,
         "| %d | %.3f | %.3f | %.3f | %.3f | %.2f |%n",
         n, time[0], time[1], time[2], time[3], speedup));

   csv.append(String.format(Locale.US, "%d,%.3f,%.3f,%.3f,%.3f%n",
         n, time[0], time[1], time[2], time[3]));
}

md.append("\n## Таблица 3. Теоретический объём памяти\n\n");
md.append("| Длина | Полная матрица, байт | Две строки, байт |\n");
md.append("|---:|---:|---:|\n");

System.out.println();
System.out.println("Таблица 3. Теоретический объём памяти (байт)");
System.out.printf("%6s %20s %20s%n", "n", "Полная матрица", "Две строки");

for (int n : SIZES) {
   long matrixBytes = (long) (n + 1) * (n + 1) * Integer.BYTES;
   long rowsBytes = 2L * (n + 1) * Integer.BYTES;
```

```
            System.out.printf("%6d %20d %20d%n", n, matrixBytes, rowsBytes);
            md.append(String.format("| %d | %d | %d |%n", n, matrixBytes, rowsBytes));
        }

        try (PrintWriter out = new PrintWriter("results.md", "UTF-8")) {
            out.print(md);
        }
        try (PrintWriter out = new PrintWriter("results.csv", "UTF-8")) {
            out.print(csv);
        }

        System.out.println();
        System.out.println("Результаты сохранены в results.md и results.csv");
    }

    /** Прогрев JIT-компилятора перед основными замерами. */
    private static void warmUp(Random random) {
        String a = randomString(random, 300);
        String b = randomString(random, 300);

        DistanceAlgorithm matrix = new LevenshteinMatrix();
        DistanceAlgorithm twoRows = new LevenshteinTwoRows();

        for (int i = 0; i < 30; i++) {
            sink += matrix.distance(a, b);
            sink += twoRows.distance(a, b);
        }

        String[] as = {a, b, a, b};
        String[] bs = {b, a, b, a};
        try (LevenshteinParallel parallel = new LevenshteinParallel(4)) {
            for (int i = 0; i < 10; i++) {
                sink += parallel.distances(as, bs)[0];
            }
        }
    }

    /** Минимальное время работы алгоритма для одной пары, мс. */
    private static double measure(DistanceAlgorithm algorithm, String a, String b) {
        for (int i = 0; i < WARMUP; i++) {
            sink += algorithm.distance(a, b);
        }

        double best = Double.MAX_VALUE;
        for (int i = 0; i < RUNS; i++) {
            long start = System.nanoTime();
            int result = algorithm.distance(a, b);
```

```
            long elapsed = System.nanoTime() - start;

            sink += result;
            best = Math.min(best, elapsed / 1_000_000.0);
        }
        return best;
    }

    /** Минимальное время обработки набора пар, мс. */
    private static double measureBatch(LevenshteinParallel algorithm,
                                       String[] as, String[] bs) {
        for (int i = 0; i < BATCH_WARMUP; i++) {
            sink += algorithm.distances(as, bs)[0];
        }

        double best = Double.MAX_VALUE;
        for (int i = 0; i < BATCH_RUNS; i++) {
            long start = System.nanoTime();
            int[] result = algorithm.distances(as, bs);
            long elapsed = System.nanoTime() - start;

            sink += result[0] + result[result.length - 1];
            best = Math.min(best, elapsed / 1_000_000.0);
        }
        return best;
    }

    private static String randomString(Random random, int length) {
        char[] chars = new char[length];
        for (int i = 0; i < length; i++) {
            chars[i] = ALPHABET.charAt(random.nextInt(ALPHABET.length()));
        }
        return new String(chars);
    }
}
```

ПРИЛОЖЕНИЕ Б

Инструкция пользователя

Для работы программы требуется JDK версии 17 или новее (проверено на JDK 21). Проверить установку можно командой javac -version в командной строке. Если команда не найдена, необходимо установить JDK и добавить его папку bin в переменную среды PATH.

Состав папки Программа:

– src/levenshtein — исходные файлы программы;

– build.bat — компиляция всех классов в папку out;

– run.bat — демонстрационный запуск;

– test.bat — запуск тестов;

– bench.bat — запуск замеров производительности.

Сборка выполняется командой:

```
javac -encoding UTF-8 -d out src\levenshtein\*.java
```

или двойным щелчком по файлу build.bat. После успешной сборки классы появляются в папке out.

Демонстрационный запуск:

```
java -cp out levenshtein.Main
```

Расчёт расстояния для конкретной пары строк:

```
java -cp out levenshtein.Main "кот" "код"
```

Запуск тестов:

```
java -cp out levenshtein.TestRunner
```

Запуск замеров (результаты сохраняются в results.md и results.csv):

```
java -Xmx1g -cp out levenshtein.Benchmark
```

Проверка предельного объёма памяти при ограничении кучи 256 МБ:

```
java -Xmx256m -cp out levenshtein.MemoryProbe matrix 8000
java -Xmx256m -cp out levenshtein.MemoryProbe rows 1000000
```

Если в консоли вместо русских букв выводятся посторонние символы, рекомендуется выполнить команду chcp 65001 перед запуском программы.

ПРИЛОЖЕНИЕ В

Примеры входных и выходных данных

Пример В.1. Демонстрационный запуск программы

```
Демонстрация работы алгоритма Левенштейна
----------------------------------------
"кот"     -> "код"     : 1 (матрица) / 1 (две строки)
"кот"     -> "скот"    : 1 (матрица) / 1 (две строки)
"дом"     -> "дым"     : 1 (матрица) / 1 (две строки)
"hello"   -> "yellow"  : 2 (матрица) / 2 (две строки)
""        -> "abc"     : 3 (матрица) / 3 (две строки)
"abc"     -> "abc"     : 0 (матрица) / 0 (две строки)
```

Пример В.2. Результат выполнения тестов

```
Таблица 2. Случайные строки: 300 пар, длина 0...200, алфавит 30 букв
Несовпадений с эталоном: 0

Таблица 3. Пакетный параллельный режим: 40 пар, длина 200...400
Несовпадений с эталоном: 0

ИТОГ: все тесты пройдены успешно.
```

Пример В.3. Фрагмент файла results.md

```
| Длина | Полная матрица | Две строки |
|---:|---:|---:|
| 1000 | 8.632 | 4.486 |
| 2000 | 31.693 | 28.079 |
| 4000 | 201.803 | 92.062 |

| Длина | 1 поток | 2 потока | 4 потока | 8 потоков | Ускорение (8) |
|---:|---:|---:|---:|---:|---:|
| 500 | 95.221 | 39.888 | 22.152 | 15.149 | 6.29 |
| 1000 | 320.192 | 179.706 | 99.088 | 61.532 | 5.20 |
| 2000 | 861.057 | 539.235 | 325.905 | 224.965 | 3.83 |
```

Пример В.4. Проверка объёма памяти при -Xmx256m

```
ОК: матрица 8001x8001 выделена
Теоретический объём: 244 МБ
```

OutOfMemoryError при n = 9000 (режим matrix)

ОК: два массива длины 1000001 выделены

Теоретический объём: 7812 КБ